Fully Homomorphic Encryption for Privacy-Preserving Computation on Pars Network

Pars Network Technical Whitepaper PNP-009

Zach KellingPars DAOresearch@pars.dev Version 1.0

2026

Abstract

We present a fully homomorphic encryption (FHE) framework for privacy-preserving computation on the Pars Network. Diaspora communities operating under surveillance require not merely encrypted storage but encrypted *computation*: the ability to search encrypted message archives, tally encrypted governance votes, and run natural language processing inference on encrypted Persian text—all without exposing plaintext to any server or validator. We instantiate two FHE schemes: TFHE for Boolean circuits enabling exact encrypted search and ballot tallying, and CKKS for approximate arithmetic enabling private neural network inference on Persian NLP models. We design a hybrid pipeline that composes both schemes via scheme switching, describe GPU acceleration strategies for bootstrapping, and provide concrete benchmarks. Integration with the Pars session layer, distributed inference framework, and governance system yields end-to-end encrypted computation pipelines resistant to both network-level and application-level surveillance.

Keywords: fully homomorphic encryption, TFHE, CKKS, encrypted search, confidential voting, private NLP inference, Persian text, GPU acceleration

Contents

1 Introduction **4**
1.1 Problem Statement . . . 4
1.2 Contributions . . . 4

2 Background **4**
2.1 FHE Generations . . . 4
2.2 TFHE . . . 5
2.3 CKKS . . . 5
2.4 Security Parameters . . . 5

3 Encrypted Messaging Search **5**
3.1 Architecture . . . 5
3.2 Equality Circuit . . . 6
3.3 Optimisations . . . 6
3.4 Performance . . . 6

4 Confidential Governance Voting **6**
4.1 Protocol . . . 7
4.2 Coercion Resistance Integration . . . 7

5 Private NLP Inference **7**
5.1 Motivation . . . 7
5.2 CKKS for Neural Networks . . . 7
5.3 Persian Text Encoding . . . 8
5.4 Inference Pipeline . . . 8
5.5 Performance . . . 8

6 Hybrid Pipeline and Scheme Switching **9**
6.1 Scheme Switching . . . 9
6.2 Pipeline Composition . . . 9

7 GPU Acceleration **9**
7.1 TFHE Bootstrapping on GPU . . . 9
7.2 CKKS on GPU . . . 10

8 Security Analysis **10**
8.1 Cryptographic Assumptions . . . 10
8.2 Side-Channel Resistance . . . 10
8.3 Post-Quantum Security . . . 10

9 Integration with Pars Infrastructure **10**
9.1 Session Layer (PNP-003) . . . 10
9.2 Governance (PNP-005) . . . 10
9.3 Distributed Inference . . . 11
9.4 Lux Network . . . 11

10 Performance Summary **11**

11 Related Work **11**

12 Conclusion **11**

1 Introduction

End-to-end encryption protects data in transit and at rest. It does not protect data *in use*. When a Pars user searches their encrypted message history, the search query and results are exposed at the application layer. When a governance vote is tallied, individual ballots must be decrypted by some authority or threshold committee. When a Persian NLP model classifies content for a user, the input text is visible to the inference server.

For the Persian diaspora, these exposure points are attack surfaces. A compromised inference node leaks the content of private queries. A compromised tally authority reveals how individuals voted—information that can endanger families inside Iran.

Fully homomorphic encryption eliminates these exposure points by enabling arbitrary computation on ciphertexts. The result, when decrypted, equals the result of the same computation on the corresponding plaintexts.

1.1 Problem Statement

Three Pars subsystems require computation on encrypted data:

1. **Encrypted messaging search.** Users must search message archives stored on Session daemons [9] without revealing queries or results to the storage nodes.
2. **Confidential governance voting.** The governance system [11] requires verifiable tallying without revealing individual votes, even to the tallying committee.
3. **Private NLP inference.** The distributed inference framework [12] serves Persian language models; users must query models without revealing input text.

1.2 Contributions

- A TFHE instantiation for Boolean-circuit computations on Pars: encrypted keyword search and encrypted ballot tallying.
- A CKKS instantiation for approximate-arithmetic computations: private inference on Persian NLP models.
- A hybrid pipeline with scheme switching between TFHE and CKKS.
- GPU acceleration for FHE bootstrapping targeting commodity hardware available to diaspora users.
- Concrete benchmarks and integration specifications for the Pars session layer (PNP-003), governance (PNP-005), and distributed inference subsystems.

2 Background

2.1 FHE Generations

Gentry's 2009 construction [1] demonstrated the theoretical possibility of FHE. Subsequent generations reduced overhead:

1. **Second generation** (BGV/BFV): Levelled schemes with batching via CRT packing. Efficient for integer arithmetic.

2. **Third generation** (GSW/FHEW/TFHE): Boolean-gate evaluation with fast bootstrapping (<10 ms per gate).

3. **Fourth generation** (CKKS): Approximate arithmetic over $\mathbb{C}$, suitable for machine learning workloads.

2.2 TFHE

TFHE [2] operates on encrypted bits. Each ciphertext encrypts a single bit under the Learning With Errors (LWE) assumption. Homomorphic NAND gates compose to arbitrary Boolean circuits. Programmable bootstrapping simultaneously reduces noise and evaluates a lookup table, enabling non-linear functions.

Definition 2.1 (TFHE Ciphertext)**.** *A TFHE ciphertext of bit* $\mu \in \{0, 1\}$ *under key* $\mathbf{s}$ *is* $(\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ *where* $b = \langle \mathbf{a}, \mathbf{s} \rangle + \mu \cdot q/4 + e$ *for small error* e*.*

2.3 CKKS

CKKS [3] encodes vectors of complex (or real) numbers into polynomial rings. Homomorphic addition and multiplication operate with bounded precision loss, analogous to floating-point arithmetic.

Definition 2.2 (CKKS Encoding)**.** *Given* $\mathbf{z} \in \mathbb{C}^{N/2}$*, the encoding maps* $\mathbf{z}$ *to a polynomial* $m(X) \in \mathbb{Z}[X]/(X^N + 1)$ *via the canonical embedding, scaled by factor* Δ*. Decoding recovers* $\mathbf{z}$ *with precision* $O(1/\Delta)$*.*

2.4 Security Parameters

All instantiations target 128-bit security under the LWE/RLWE assumptions, following the Homomorphic Encryption Standard [4]. We select parameters from Table 1.

Table 1: FHE parameter sets for Pars applications.

Scheme	n	$\log q$	**Slots**	**Use**
TFHE	630	32	1	Search, voting
CKKS	16384	438	8192	NLP inference

3 Encrypted Messaging Search

3.1 Architecture

Pars Session daemons (PNP-003) store encrypted message archives. We extend the storage layer with an encrypted search index.

1. The user encrypts each message keyword under their TFHE key and uploads the encrypted index alongside the ciphertext message.

2. To search, the user encrypts the query keyword under the same key and submits it to the daemon.

3. The daemon homomorphically evaluates an equality circuit between the encrypted query and each encrypted index entry.
4. The daemon returns encrypted match indicators; the user decrypts locally.

3.2 Equality Circuit

Definition 3.1 (Encrypted Equality)**.** *Given encrypted keywords* $\mathrm{Enc}(w_1), \mathrm{Enc}(w_2)$*, each represented as* ℓ*-bit strings, the equality circuit computes:*

$$\mathrm{Enc}(\mathsf{EQ}) = \mathrm{Enc}\left(\bigwedge_{i=1}^{\ell} \neg(w_{1,i} \oplus w_{2,i})\right)$$

For ℓ-bit keywords, the circuit requires ℓ XNOR gates and $\ell - 1$ AND gates, totalling $2\ell - 1$ bootstrapped gates.

Theorem 3.1 (Search Complexity)**.** *Searching* m *encrypted keywords of* ℓ *bits each requires* $m(2\ell - 1)$ *bootstrapped TFHE gates. With bootstrapping time* t_{bs}*, total wall-clock time is* $m(2\ell - 1) \cdot t_{\mathrm{bs}}/p$ *on* p *parallel cores.*

3.3 Optimisations

Bloom Filter Pre-screening. Each message is associated with an encrypted Bloom filter. The daemon first evaluates the Bloom filter circuit (smaller than full equality) to prune non-matching messages before running full equality.

Batched Bootstrapping. Multiple TFHE gates sharing the same bootstrapping key can batch FFT operations on the GPU, reducing amortised cost.

3.4 Performance

Table 2: Encrypted search benchmarks (1000 messages, 128-bit keywords).

Configuration	**Time (s)**	**Throughput (msg/s)**
CPU, 1 core	127.5	7.8
CPU, 16 cores	8.2	121.9
GPU (RTX 4090)	1.4	714.3

4 Confidential Governance Voting

The Pars governance system (PNP-005) requires that individual votes remain secret while the tally is publicly verifiable. FHE provides a cleaner solution than mix-nets or homomorphic tallying under ElGamal: votes are encrypted under a common FHE key, and the tally is computed entirely in the encrypted domain.

4.1 Protocol

1. **Setup.** The governance committee generates a threshold TFHE key $(pk, \{sk_i\}_{i=1}^{t})$ using the threshold signing infrastructure (PNP-010). Any t-of-n committee members can jointly decrypt.

2. **Vote casting.** Each voter encrypts their ballot $v_j \in \{0,1\}^k$ (for k candidates) under pk and submits to the governance contract with a ZK proof of well-formedness (PNP-006).

3. **Tallying.** The contract homomorphically sums all encrypted ballots:

$$\mathrm{Enc}(T) = \bigoplus_{j=1}^{N} \mathrm{Enc}(v_j)$$

   where $\oplus$ denotes the TFHE addition circuit.

4. **Decryption.** The committee jointly decrypts the encrypted tally $\mathrm{Enc}(T) \to T$ using threshold decryption. Only the aggregate is revealed.

Theorem 4.1 (Vote Privacy)**.** *Under the LWE assumption, no coalition of fewer than t committee members can determine any individual vote v_j from the encrypted ballots and the decrypted tally T.*

Proof. Each $\mathrm{Enc}(v_j)$ is semantically secure under LWE. The homomorphic sum preserves semantic security: the tally ciphertext is computationally indistinguishable from an encryption of any other valid tally. Threshold decryption reveals only T, which information-theoretically determines a set of valid vote vectors but not the individual mapping. □

4.2 Coercion Resistance Integration

The coercion resistance protocols (PNP-005) compose with FHE voting:

- **Panic ballots.** A coerced user submits a ballot under the panic key; the system re-randomises it to match the decoy profile.

- **Receipt-freeness.** Since the voter never sees the encrypted ballot after submission (it is re-randomised by the contract), no receipt can be produced.

5 Private NLP Inference

5.1 Motivation

The Pars distributed inference framework provides Persian language models for translation, summarisation, and content classification. Users in censored environments must query these models without revealing input text to inference nodes.

5.2 CKKS for Neural Networks

Neural network inference consists of matrix multiplications, additions, and non-linear activations. CKKS handles the linear operations natively. Non-linear activations (ReLU, GELU) are approximated by low-degree polynomials.

Definition 5.1 (Polynomial Activation)**.** *The GELU activation $\sigma(x) = x \cdot \Phi(x)$ is approximated by a degree-d minimax polynomial $p_d(x)$ on $[-B, B]$:*

$$\max_{x \in [-B,B]} |p_d(x) - \sigma(x)| < \epsilon$$

For $d = 7$, $B = 8$, $\epsilon < 10^{-3}$.

5.3 Persian Text Encoding

Persian text presents specific challenges for FHE:

1. **Right-to-left script.** Token embeddings must preserve RTL ordering within CKKS slots.
2. **Large vocabulary.** The SentencePiece vocabulary for Persian contains 32K tokens; embedding lookup is implemented as encrypted matrix-vector multiplication.
3. **Diacritics and Ezafe.** Subword tokenisation must handle optional diacritical marks without information leakage through token count.

5.4 Inference Pipeline

1. User encodes input tokens as one-hot vectors in CKKS slots.
2. User encrypts the packed vector under their CKKS public key.
3. The inference node applies the model weights (in plaintext) to the encrypted input via homomorphic matrix multiplication.
4. Between layers, the node performs rescaling and applies polynomial activations.
5. The final logits (encrypted) are returned to the user for local decryption and argmax.

Theorem 5.1 (Inference Privacy)**.** *The inference node learns nothing about the input text, intermediate activations, or output logits beyond what is revealed by the ciphertext size and number of evaluation rounds.*

5.5 Performance

Table 3: Private inference benchmarks (4-layer transformer, 128 tokens).

Configuration	**Latency (s)**	**Accuracy Drop**
Plaintext baseline	0.02	—
CKKS, CPU	142.3	1.2%
CKKS, GPU	18.7	1.2%

The 1.2% accuracy drop results from polynomial activation approximation and CKKS precision limits. For classification tasks (sentiment, topic), this is acceptable.

6 Hybrid Pipeline and Scheme Switching

Some applications require both exact (TFHE) and approximate (CKKS) computation. For example, a governance proposal may require exact vote tallying (TFHE) combined with approximate sentiment analysis of discussion text (CKKS).

6.1 Scheme Switching

Definition 6.1 (TFHE-to-CKKS Switch)**.** *Given a TFHE ciphertext c_{TFHE} encrypting bit μ, the scheme switch produces a CKKS ciphertext c_{CKKS} encrypting $\mu \in \{0.0, 1.0\}$ under a compatible CKKS key, via:*

1. *Functional bootstrap c_{TFHE} to extract μ into an RLWE ciphertext.*
2. *Key-switch from the bootstrapping key space to the target CKKS key.*
3. *Rescale to the CKKS encoding precision.*

The reverse switch (CKKS-to-TFHE) applies sign extraction followed by modulus switching.

6.2 Pipeline Composition

1. Encrypted search (TFHE) identifies matching messages.
2. Scheme switch converts match indicators to CKKS.
3. CKKS inference runs sentiment classification on matched messages.
4. Results returned encrypted; user decrypts locally.

7 GPU Acceleration

FHE bootstrapping dominates computation cost. We target GPU acceleration on two fronts:

7.1 TFHE Bootstrapping on GPU

The TFHE bootstrap operation consists of:

1. Blind rotation: N external products, each an NTT-based polynomial multiplication.
2. Key switching: matrix-vector multiplication in $\mathbb{Z}_q$.

Both operations parallelise across GPU threads. We adapt the cuFHE approach [5] with the following modifications for Pars:

- Batch multiple independent gates into a single kernel launch.
- Use shared memory for the bootstrapping key (fits in 48 KB for $n = 630$).
- Pipeline NTT with butterfly stages across warps.

Theorem 7.1 (GPU Speedup)**.** *For m independent TFHE gates, GPU bootstrapping achieves speedup $S = \min(m, C) \cdot \alpha$ over single-core CPU, where C is the GPU core count and $\alpha \approx 3$–$5\times$ is the per-gate NTT speedup from dedicated hardware.*

7.2 CKKS on GPU

CKKS homomorphic multiplication requires two NTTs on degree-N polynomials. For $N = 16384$, each NTT processes 2^{14} elements—a natural fit for GPU parallelism.

The Pars GPU pipeline (PNP-011) provides the NTT infrastructure shared with post-quantum cryptography (PNP-004).

8 Security Analysis

8.1 Cryptographic Assumptions

[Ring-LWE] For dimension N, modulus q, and Gaussian error distribution χ, the Ring-LWE problem is computationally hard: no polynomial-time algorithm distinguishes $(a, a \cdot s + e)$ from uniform over $R_q \times R_q$, where $R_q = \mathbb{Z}_q[X]/(X^N + 1)$.

Both TFHE and CKKS security reduce to Ring-LWE. Our parameters follow the Homomorphic Encryption Standard [4], targeting $\lambda \geq 128$ bits.

8.2 Side-Channel Resistance

FHE evaluation on untrusted nodes must resist timing and power side-channels:

- **Constant-time NTT.** All polynomial operations use constant-time butterfly networks.
- **Oblivious memory access.** Bootstrapping key access patterns are data-independent.
- **Ciphertext padding.** All ciphertexts are padded to uniform size to prevent length-based information leakage.

8.3 Post-Quantum Security

Both TFHE and CKKS are based on lattice problems and are believed to resist quantum attacks. The parameter sets in Table 1 provide 128-bit post-quantum security under the methodology of PNP-004 [10].

9 Integration with Pars Infrastructure

9.1 Session Layer (PNP-003)

The encrypted search index is stored alongside E2E-encrypted messages on Session daemons. The TFHE public key is distributed via the same prekey bundle mechanism used for session establishment.

9.2 Governance (PNP-005)

FHE voting replaces the current mix-net approach for ballot tallying. The threshold TFHE key is managed by the governance committee using the threshold signing protocol (PNP-010).

9.3 Distributed Inference

The Pars distributed inference framework currently exposes plaintext queries to inference nodes. CKKS inference wraps the existing model serving pipeline: the model weights remain in plaintext on the node, but all user data is encrypted.

9.4 Lux Network

Pars inherits FHE research from the Lux ecosystem (lux-tfhe), sharing parameter sets and GPU kernels. Lux's FHE-enabled confidential smart contracts provide the L1 settlement layer for Pars FHE computations.

10 Performance Summary

Table 4: End-to-end latency for FHE applications on Pars.

Application	**CPU (s)**	**GPU (s)**	**Overhead**
Search (1K msgs)	8.2	1.4	100×
Vote tally (10K)	24.1	3.8	190×
NLP inference (128 tok)	142.3	18.7	935×

The overhead relative to plaintext computation is substantial but acceptable for the security properties gained. Encrypted search at 1.4 s GPU latency is within interactive thresholds. Vote tallying is a batch operation with no real-time constraint. NLP inference at 18.7 s is viable for asynchronous queries (translation, summarisation) though not for real-time conversation.

11 Related Work

Concrete. Zama's Concrete framework [6] provides a TFHE compiler for Rust and Python. Pars builds on the same TFHE parameter sets but adds diaspora-specific optimisations (Persian text handling, mesh-network key distribution).

SEAL. Microsoft SEAL [7] implements BFV and CKKS. Pars uses CKKS parameters compatible with SEAL but deploys on decentralised inference nodes rather than centralised cloud.

HELib. IBM's HELib [8] pioneered practical BGV. Pars requires TFHE's gate-level bootstrapping for exact circuits (search, voting), which HELib does not support.

Lux-TFHE. The Lux network's FHE infrastructure provides shared NTT kernels and parameter sets. Pars extends these with application-layer protocols specific to diaspora use cases.

12 Conclusion

We have presented a comprehensive FHE framework for the Pars Network that enables encrypted search over messaging archives, confidential governance vote tallying, and private NLP inference on

Persian text. By combining TFHE for exact circuits with CKKS for approximate arithmetic, and accelerating both via GPU, we achieve practical latencies for all three applications.

The integration with Pars's existing session layer, governance protocols, and distributed inference framework provides end-to-end encrypted computation pipelines. No plaintext user data is exposed to any untrusted node at any point in the computation.

Future work includes: (1) multi-key FHE to eliminate the threshold key setup, (2) FHE-friendly model architectures that reduce multiplicative depth, and (3) client-side hardware acceleration for mobile devices commonly used by diaspora communities.

References

[1] C. Gentry, "Fully Homomorphic Encryption Using Ideal Lattices," *STOC*, pp. 169–178, 2009.

[2] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "TFHE: Fast Fully Homomorphic Encryption over the Torus," *Journal of Cryptology*, vol. 33, pp. 34–91, 2020.

[3] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic Encryption for Arithmetic of Approximate Numbers," *ASIACRYPT*, pp. 409–437, 2017.

[4] M. Albrecht et al., "Homomorphic Encryption Standard," *HomomorphicEncryption.org*, 2018.

[5] W. Dai and B. Sunar, "cuFHE: A GPU Library for Fully Homomorphic Encryption," *IACR ePrint 2017/007*, 2018.

[6] Zama, "Concrete: TFHE Compiler for Secure Computation," 2022.

[7] Microsoft Research, "Microsoft SEAL (release 4.0)," 2022.

[8] S. Halevi and V. Shoup, "Algorithms in HElib," *CRYPTO*, pp. 554–571, 2014.

[9] Cyrus Pahlavi, Pars Team, "PNP-003: Pars Session Protocol," 2026.

[10] Cyrus Pahlavi, Pars Team, "PNP-004: Post-Quantum Cryptographic Standards for Pars Network," 2026.

[11] Cyrus Pahlavi, Pars Team, "PNP-005: Coercion-Resistant Protocols for Governance Under Authoritarian Surveillance," 2026.

[12] Cyrus Pahlavi, Pars Team, "Distributed Inference for Persian Language Models on Pars Network," 2026.